

12-docker-realtime.md

Docker CI/CD Pipeline — ELI5 + Real World Map 🗺️

🏠 Real World Analogy: Zomato Food Factory

Think of your app like a **Zomato restaurant** and Jenkins is the **Kitchen Manager** who makes sure every dish (code) is:

Written → Checked for quality → Checked for bad ingredients → Packed → Delivered

🗺️ The Big Picture Map

Developer pushes code

↓

Jenkins wakes up

↓

1. Clean Workspace	→ Sweep the kitchen
2. Pull Code	→ Get the recipe
3. SonarQube	→ Taste test
4. Quality Gate	→ Pass/Fail check
5. npm install	→ Buy ingredients
6. OWASP	→ Check expiry
7. Trivy FS scan	→ Check raw food
8. Docker Build	→ Cook the dish
9. Docker Push	→ Send to fridge
10. Trivy Image	→ Final inspection
11. Docker Run	→ Serve to customer

👤 Each Stage — ELI5 + Syntax

🔧 Stage 1 — Clean Workspace

Real world: Before cooking, clean the kitchen. No leftover mess from yesterday.

groovy

```
stage('Clean') {
    steps {
        cleanWs()    // delete all old files
    }
}
```

```
}
```

Stage 2 — Pull Code (Git)

Real world: Get today's recipe from the head chef (GitHub).

groovy

```
stage('Code') {
    steps {
        git branch: 'main', url: 'https://github.com/your-repo.git'
        //           ↑ which branch           ↑ where is the code
    }
}
```

Stage 3 — SonarQube Analysis

Real world: A food critic tastes the dish and says *"this has too much salt"* (bugs/bad code).

groovy

```
stage('Sonarqube Analysis') {
    steps {
        withSonarQubeEnv('mysonar') { // use sonar server named 'mysonar'
            sh '''
                $SCANNER_HOME/bin/sonar-scanner \
                -Dsonar.projectName=zomato \    // project display name
                -Dsonar.projectKey=zomato      // unique ID in sonar
            '''
        }
    }
}
```

Stage 4 — Quality Gate

Real world: The restaurant manager says *"if food critic gives below 3 stars, DON'T serve it."*

groovy

```
stage('Quality Gates') {
    steps {
        script {
            waitForQualityGate abortPipeline: false
        }
    }
}
```

```

        //                                     ↑ false = don't stop pipeline
even if it fails
        //                                     true  = stop pipeline if
quality fails
    }
}
}

```

Stage 5 — npm install

Real world: Order all the ingredients (libraries) needed to cook the dish.

groovy

```

stage('Install Dependencies') {
    steps {
        sh 'npm install' // reads package.json and downloads everything
    }
}

```

Stage 6 — OWASP Scan

Real world: Check if any ingredient is **expired or poisonous** (known security vulnerabilities in packages).

groovy

```

stage('OWASP') {
    steps {
        dependencyCheck additionalArguments: '--scan ./', odcInstallation:
'Dp-Check'
        //                                     ↑ scan current folder
        dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
        //                                     ↑ publish the report in Jenkins UI
    }
}

```

Stage 7 — Trivy File Scan

Real world: Health inspector checks the **raw ingredients** before cooking even starts.

groovy

```
stage('Trivy Scan') {
    steps {
        sh "trivy fs . > trivyfs.txt"
        //          ↑ scan filesystem  ↑ save result to file
    }
}
```

Stage 8 — Docker Build

Real world: Actually **cook the dish** using the recipe (Dockerfile).

groovy

```
stage('Build Dockerfile') {
    steps {
        sh 'docker build -t image1 .'
        //          ↑ tag/name  ↑ use Dockerfile in current folder
    }
}
```

Stage 9 — Docker Push

Real world: Send the cooked dish to the **central storage fridge** (DockerHub) so any branch can pick it up.

groovy

```
stage('Docker Build & Push') {
    steps {
        script {
            withDockerRegistry(credentialsId: 'docker-password') {
                // ↑ login to DockerHub using saved Jenkins credentials
                sh "docker tag image1 yourname/repo:tag"
                //          ↑ rename local image to hub format
                sh "docker push yourname/repo:tag"
                // ↑ upload to DockerHub
            }
        }
    }
}
```

Stage 10 — Trivy Image Scan

Real world: Final health check on the **packed dish** before delivery. Checks the container for CVEs.

groovy

```
stage('Scan Image') {
    steps {
        sh 'trivy image yourname/repo:tag'
        // scans the built Docker image for vulnerabilities
    }
}
```

Stage 11 — Deploy (Docker Run)

Real world: Deliver the dish to the customer's table (run container, expose port).

groovy

```
stage('Deploy') {
    steps {
        sh 'docker run -d --name zomato -p 3000:3000 yourname/repo:tag'
        //           ↑ background ↑ container name ↑ port mapping
    }
}
```

Remember It As One Line

Clean → Code → Sonar → Gate → Install → OWASP → Trivy FS → Build → Push → Trivy Image → Run

Or the food way:

Clean kitchen → Get recipe → Taste test → Pass/Fail → Buy ingredients → Check expiry → Inspect raw food → Cook → Store → Final check → Serve

Quick Cheatsheet — Tools & Why

Tool	What it does	Real world
Jenkins	Runs the whole pipeline	Kitchen Manager
SonarQube	Code quality bugs	Food critic
OWASP DC	Dependency vulnerabilities	Expiry date checker
Trivy	Container/file security scan	Health inspector

Tool	What it does	Real world
DockerHub	Image registry	Central fridge
cleanWs()	Delete old workspace	Sweep kitchen

🔑 One Gotcha You'll Always Hit

bash

```
# If Jenkins can't talk to Docker → permission error
```

```
chmod 777 /var/run/docker.sock
```

```
#           ↑ this is the "door" between Jenkins and Docker daemon
```

Like giving the kitchen manager a key to the fridge he was locked out of.